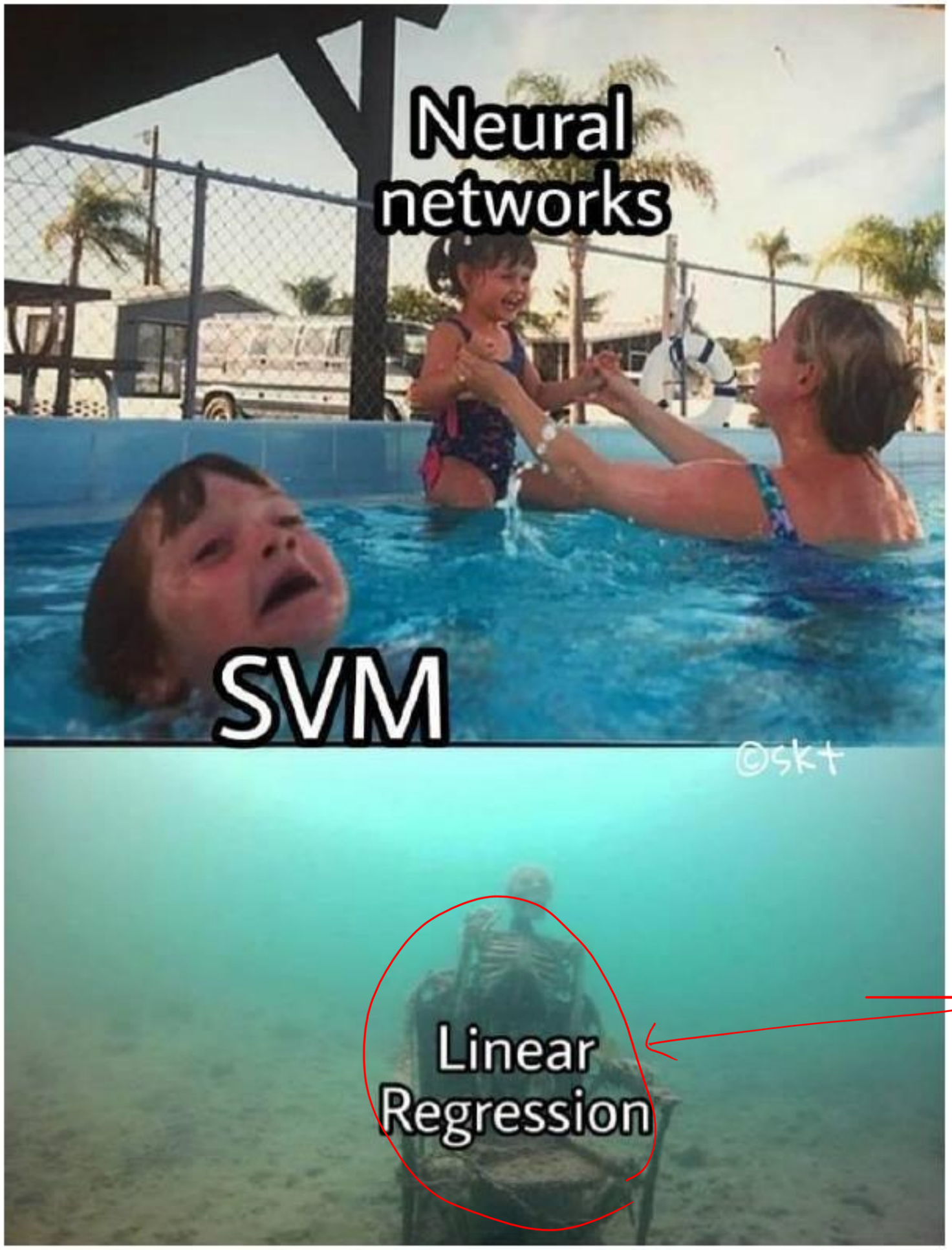
Machine Learning CS 4641-7641

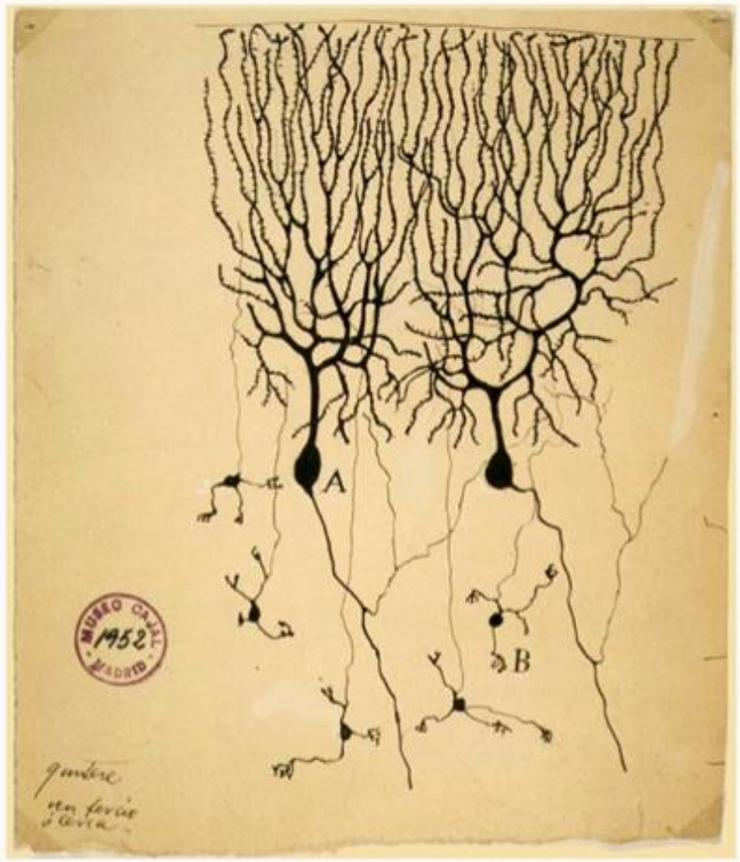
Neural Networks Forward Pass and Back Propagation

Nimisha Roy
Georgia Tech

Slides adapted from Mahdi Roozbahani


Neural networks
SVM
©skt
Linear Regression
LF = xθ

Inspiration from Biological Neurons



The first drawing of a brain cells by Santiago Ramón y Cajal in 1899

Neurons: core components of brain and the nervous system consisting of

1. Dendrites that collect information from other neurons
2. An axon that generates outgoing spikes

Neural Networks

- A robust approach for approximating **real-valued, discrete-valued or vector valued** functions

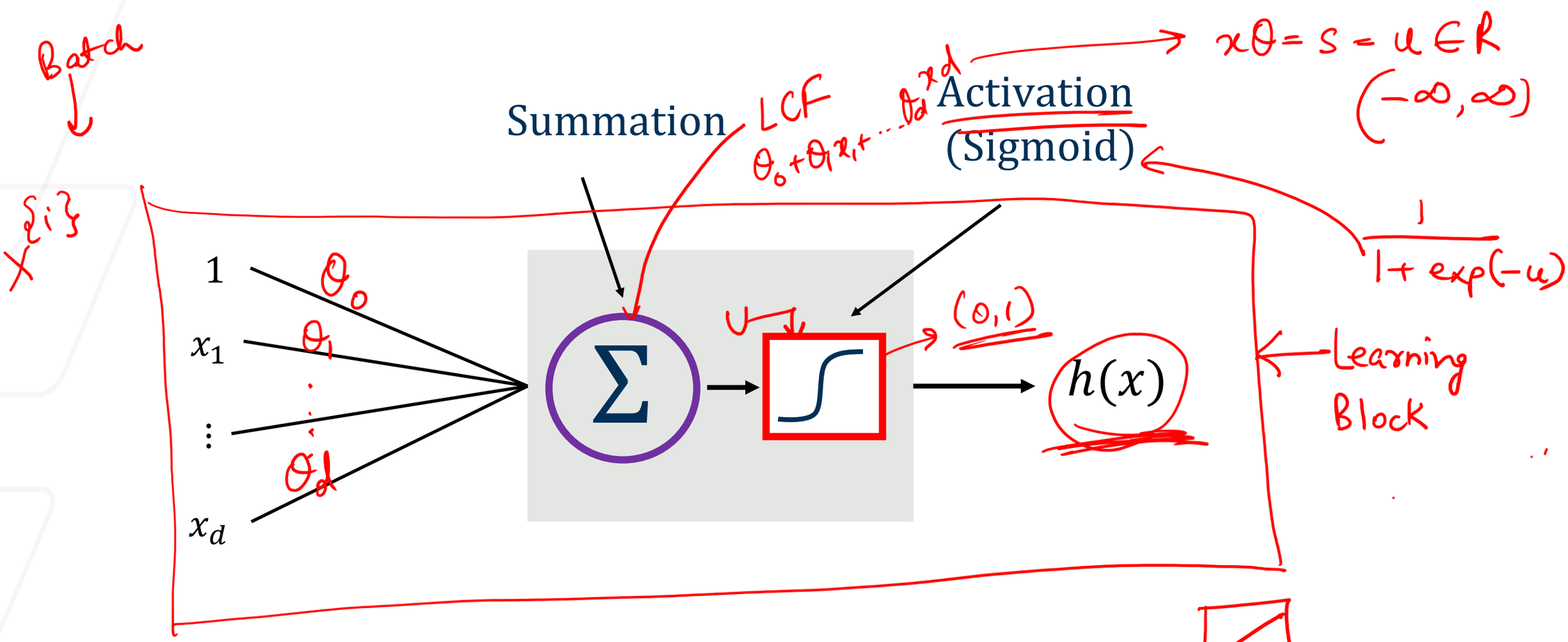
- Among the most effective **general purpose** supervised learning methods

- Apt at modeling **complex and hard to interpret data** such as images and audio

- The **backpropagation algorithm** for neural networks has been shown successful in many practical problems:

  - Handwritten character recognition, speech recognition, object recognition, some NLP problems

Georgia Tech



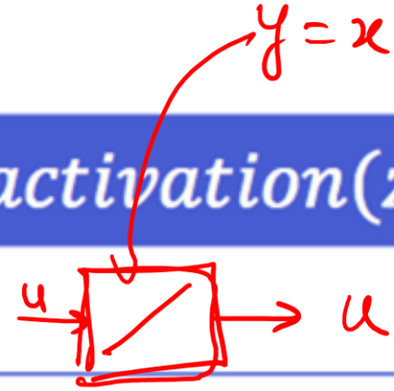
$$output = activation(x\theta + b)$$

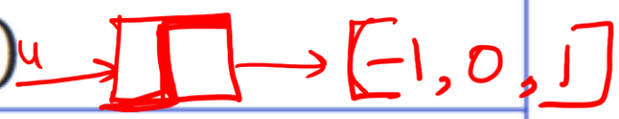
$$h(x) = o(x\theta = u) \rightarrow \text{logistic regression}$$

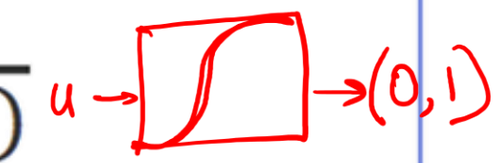
$x\theta$ = LCF = u

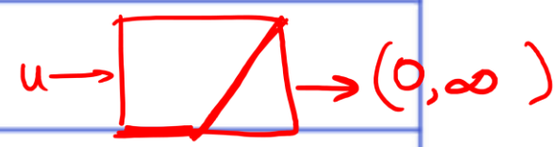
Activation Function → Introduce some kind of non-linearity in our model

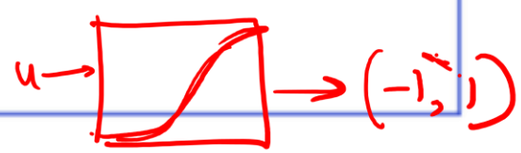
Name of the neuron	Activation function: $activation(z)$
Linear unit	z
Threshold/sign unit	$\text{sgn}(z)$
Sigmoid unit	$\frac{1}{1+\exp(-z)}$
Rectified linear unit (ReLU)	$\max(0, z)$
Tanh unit	$\tanh(z)$











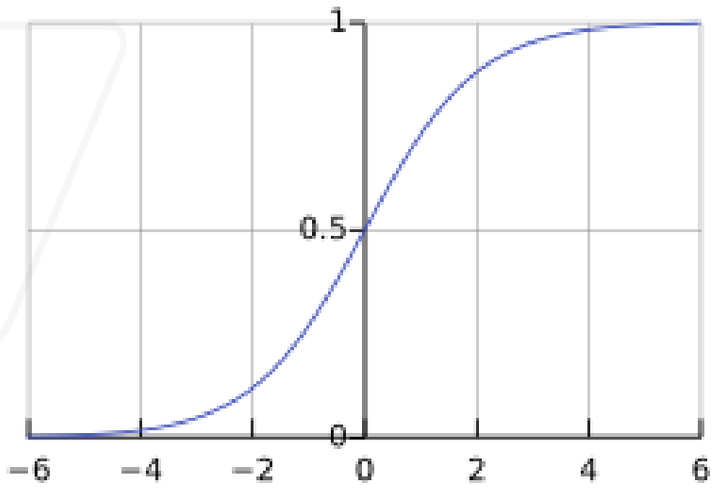
Activation function should introduce non-linearity

Very very Popular

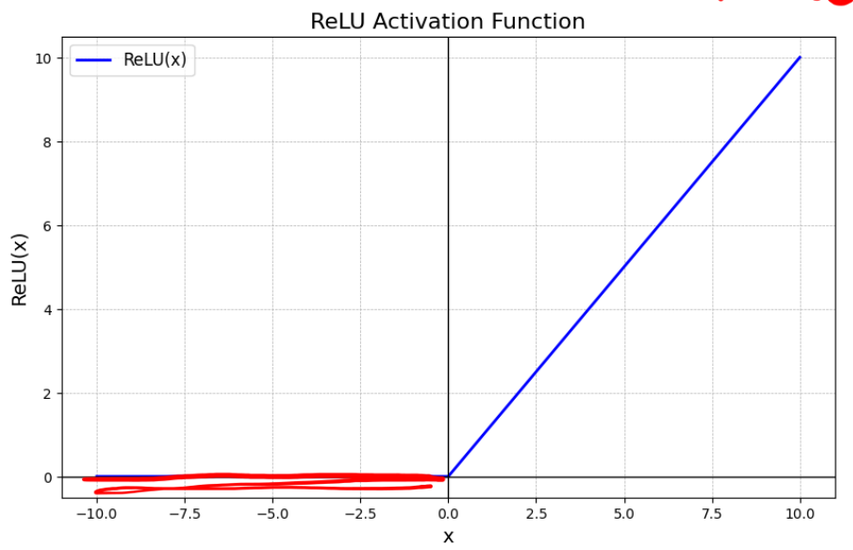
- simple
- converge very quickly

Different activation functions: o

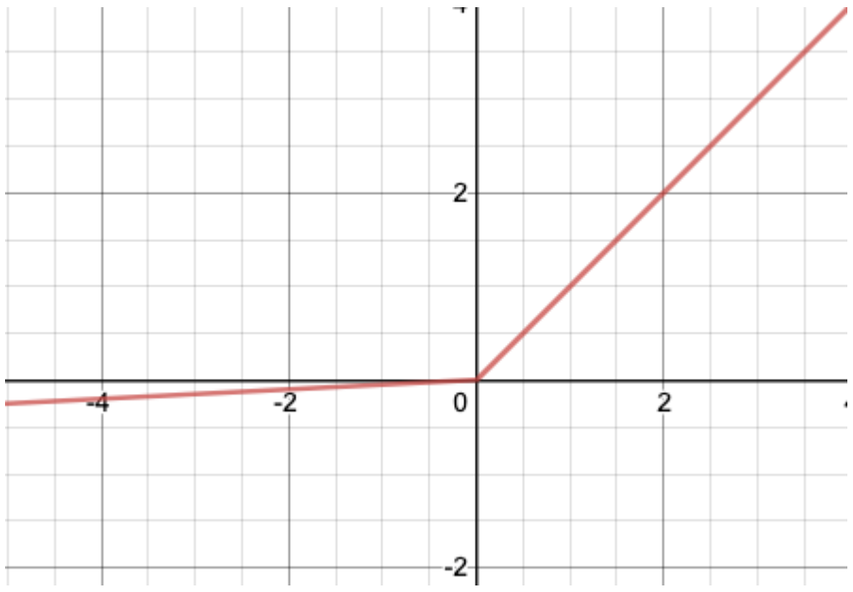
Sigmoid



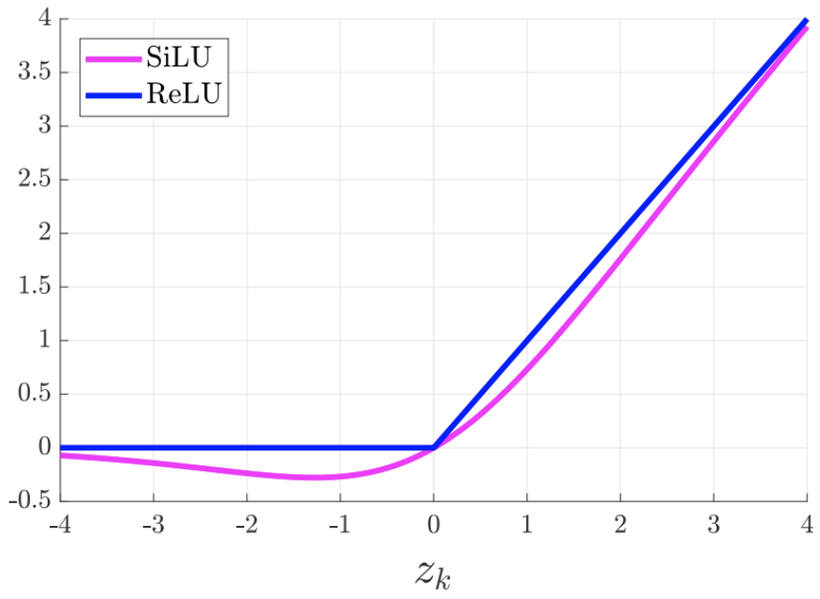
ReLu



Leaky ReLu

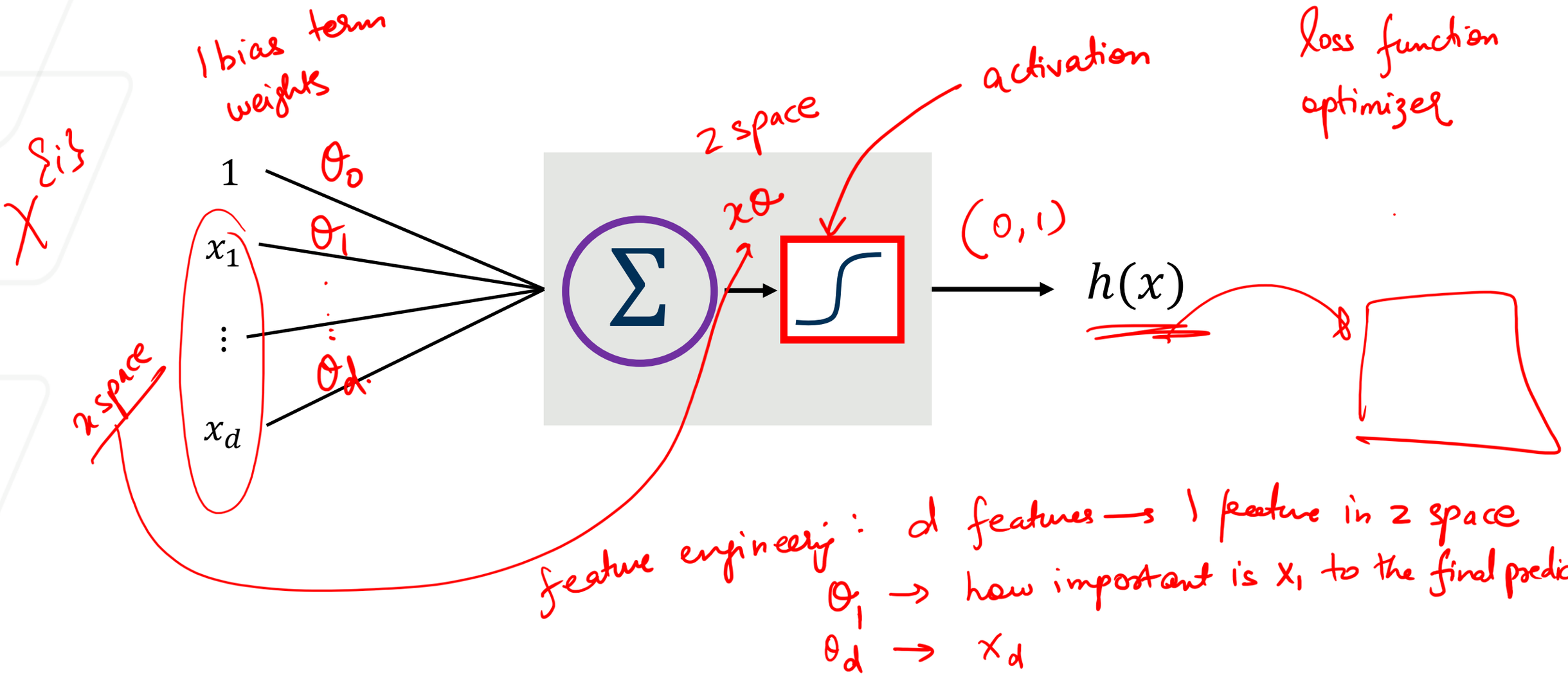


SiLu = Sigmoid + ReLu



Georgia Tech

In Neural Network, 1 learning block is 1 building block



A **neural network** is essentially a **stack (or composition)** of many **learning blocks**, where:

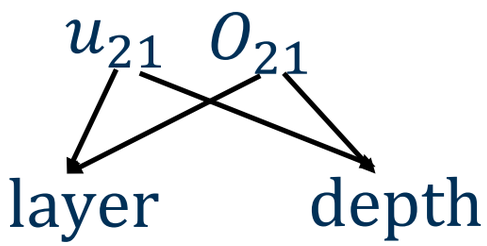
$$\textit{Output of Block } i \;=\; \textit{Input to Block } (i+1)$$

Building a Network: NN Regression

$u_{11} = \theta_0 \cdot 1 + \theta_2 \cdot x_i$

$u_{12} = \theta_1 + \theta_3 \cdot x_i$

2 building blocks



$u_{21} = \theta_4 \cdot 1 + \theta_5 \cdot O_{11} + \theta_6 \cdot O_{12}$

Input layer

1st hidden layer

Output layer

Neuron or node

feature x_i

θ_0

θ_1

θ_2

θ_3

weight

Summation

u_{11}

u_{12}

Summation output

$\frac{1}{1+\exp(-u)}$

Activation (Sigmoid)

Bias

1

O_{11}

O_{12}

(0, 1)

Sigmoid output

(0,1)

θ_4

θ_5

θ_6

2nd input layer
z space

u_{21}

Regression

O_{21}

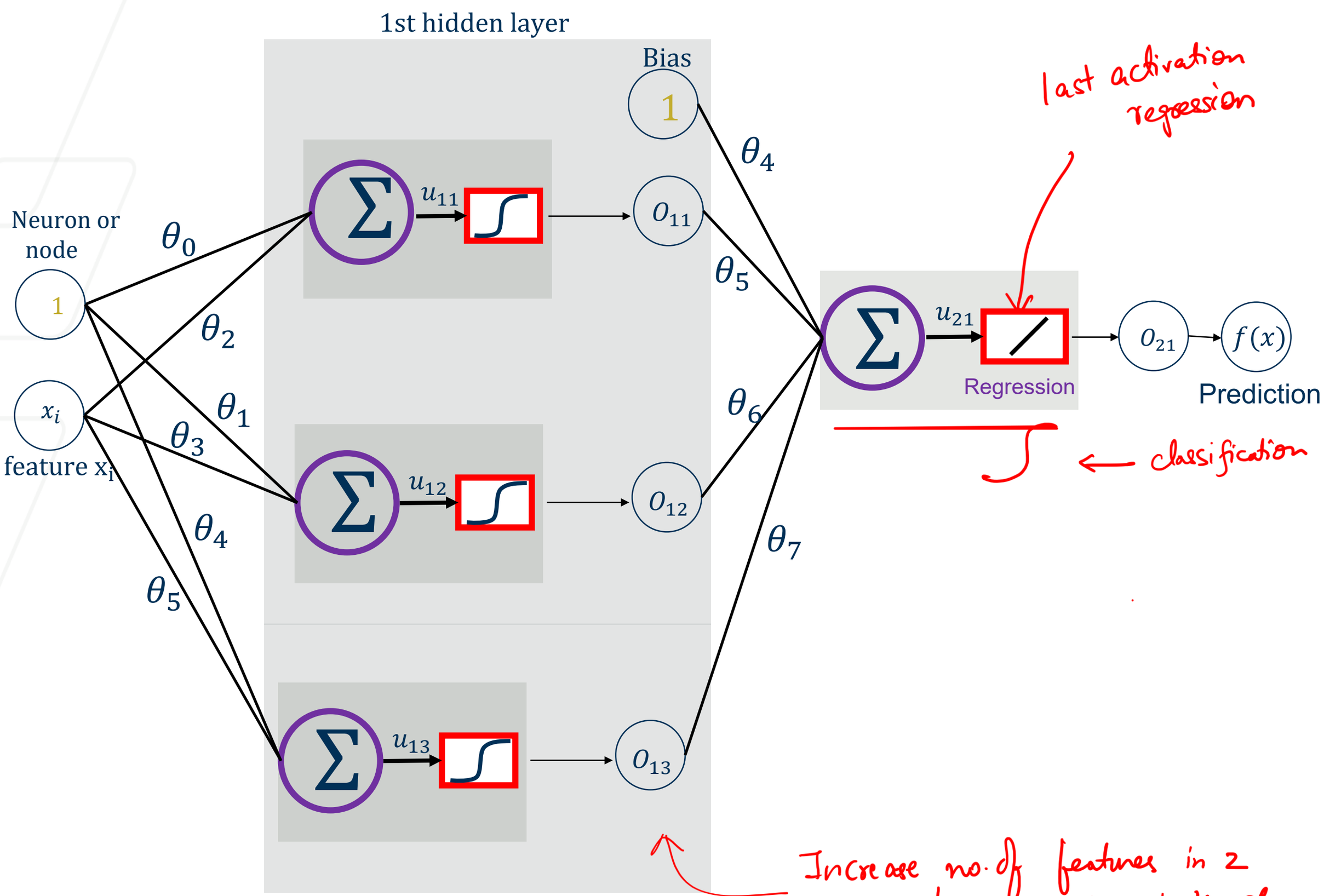
$f(x)$

Prediction

$O_{21} = u_{21} = f(z)$

input features are always fully connected to the first layer

Georgia Tech


1st hidden layer
Bias
1
Neuron or node
1
x_i
feature x_i
θ_0
θ_1
θ_2
θ_3
θ_4
θ_5
θ_4
θ_5
θ_6
θ_7
u_{11}
u_{12}
u_{13}
o_{11}
o_{12}
o_{13}
u_{21}
o_{21}
$f(x)$
Regression
Prediction
last activation regression
classification
Increase no. of features in 2 space by increasing depth of the previous layer

1st hidden layer

2nd hidden layer

Bias

Bias

Neuron or node

feature x_i

1

x_i

u_{11}

u_{12}

u_{13}

u_{21}

u_{22}

u_{23}

O_{11}

O_{12}

O_{13}

O_{21}

O_{22}

O_{23}

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22

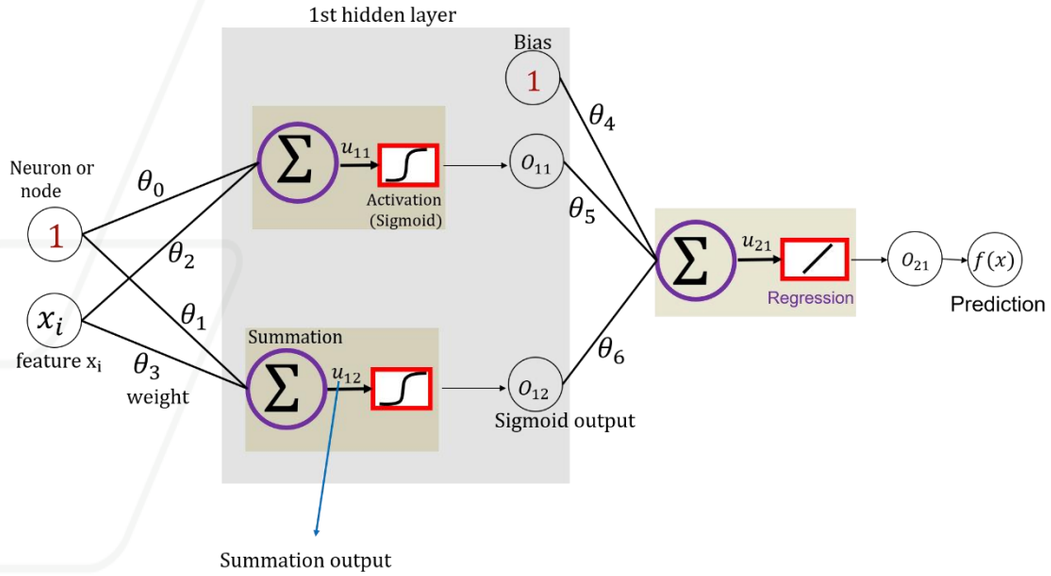
$2 \times 3 = 6$

$4 \times 3 = 12$

$4 \times 1 = 4$

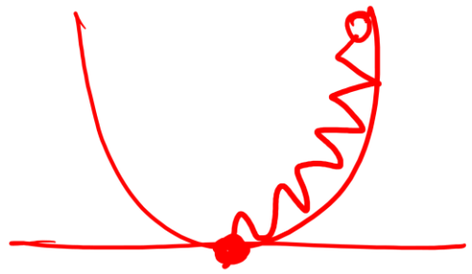
22 parameters

Georgia Tech



Gradient Descent

$$\theta^{t+1} = \theta^{t} - \alpha \cdot \frac{\partial L}{\partial \theta}$$



Step0 : Initialize θ's. Do not initialize θ to 0.

Step1 : Calculate u's & o's.
(u's ↓ θ, o's ↓ activation function)

FORWARD PASS

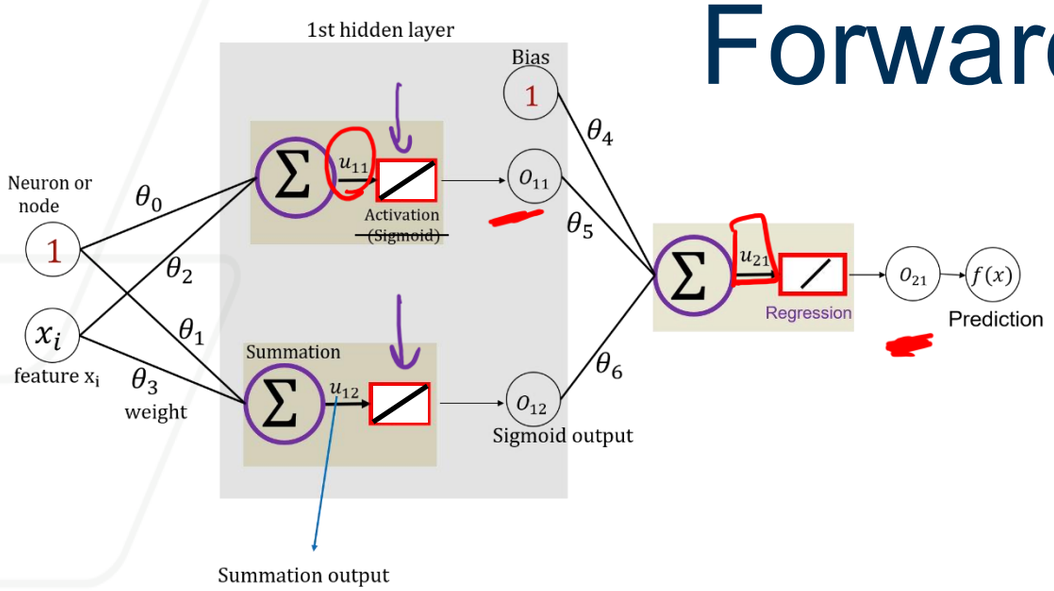
Step3 : Recalculate θ^{t+1}.

BACK PROPAGATION

Repeat Until Convergence.

Regression — MSE
Classification — CE
— CE with softmax

Forward Propagation: Rigid Network



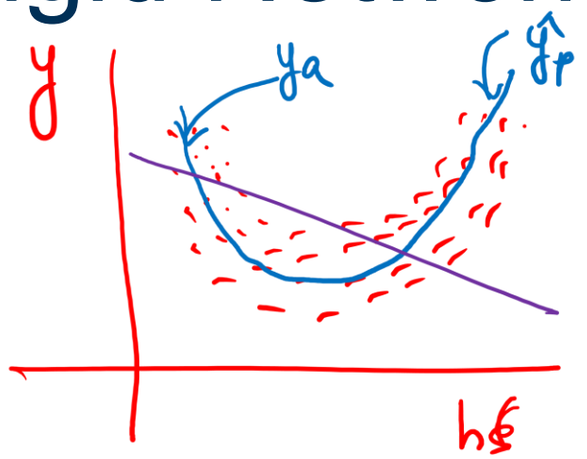
$$u_{11} = \theta_0 + \theta_2 x_i$$

$$o_{11} = u_{11}$$

$$u_{12} = \theta_1 + \theta_3 x_i$$

u_{12}: 1st layer, 2nd block

$$o_{12} = u_{12}$$



$$u_{21} = \theta_4 + \theta_5 \cdot o_{11} + o_{12} \cdot \theta_6$$

$$= \theta_4 + \theta_5(\theta_0 + \theta_2 x_i) + \theta_6(\theta_1 + \theta_3 x_i)$$

$$u_{21} = (\theta_4 + \theta_5\theta_0 + \theta_6\theta_1) + x_i(\theta_5\theta_2 + \theta_6\theta_3)$$

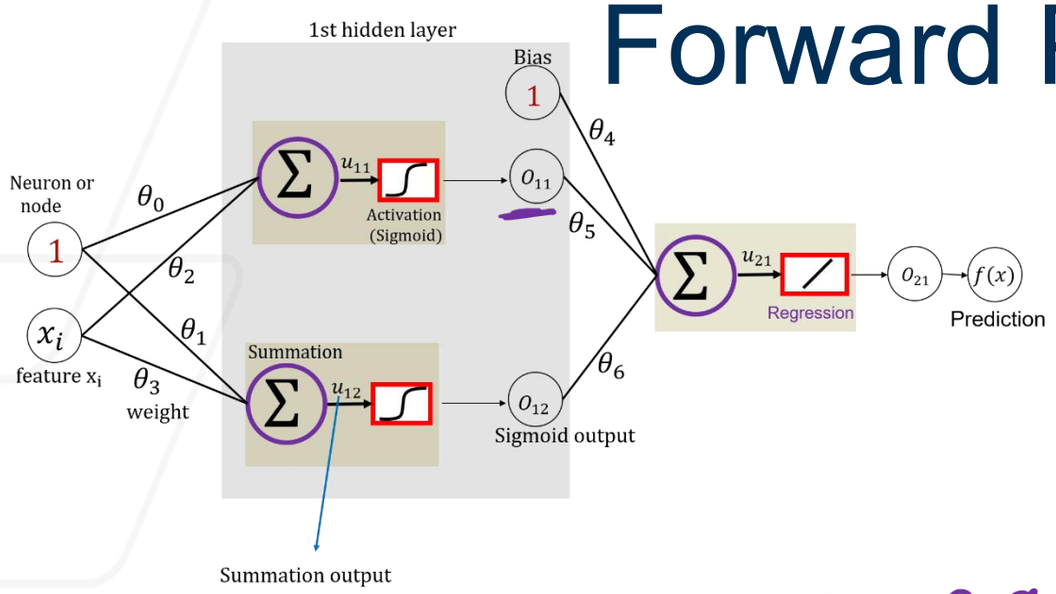
$$o_{21} = u_{21} = f(x)$$

$\theta_0 + x_i \theta_1$

θ_1: slope or rotation

θ_0: y intercept

Forward Propagation: Flexible Network



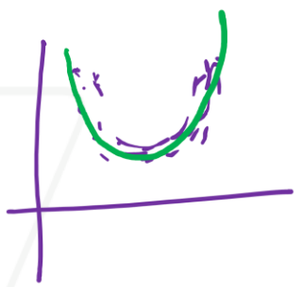
$$u_{11} = \theta_0 + \theta_2 x_i$$

$$o_{11} = \frac{1}{1 + \exp(-u_{11})} = \frac{1}{1 + \exp(-\theta_0 - \theta_2 x_i)}$$

horizontal shrinking or stretching

horizontal translation

0.5



$$u_{12} = \theta_1 + \theta_3 x_i$$

$$o_{12} = \frac{1}{1 + \exp(-\theta_1 - \theta_3 x_i)}$$

$$u_{21} = \theta_4 + \theta_5 o_{11} + \theta_6 o_{12}$$

$$= \theta_4 + \frac{\theta_5}{1 + \exp(-\theta_0 - \theta_2 x_i)} + \frac{\theta_6}{1 + \exp(-\theta_1 - \theta_3 x_i)}$$

$$o_{21} = f(x) = u_{21}$$

vertical translation

vertical shrinky or stretchy

horizontal translation

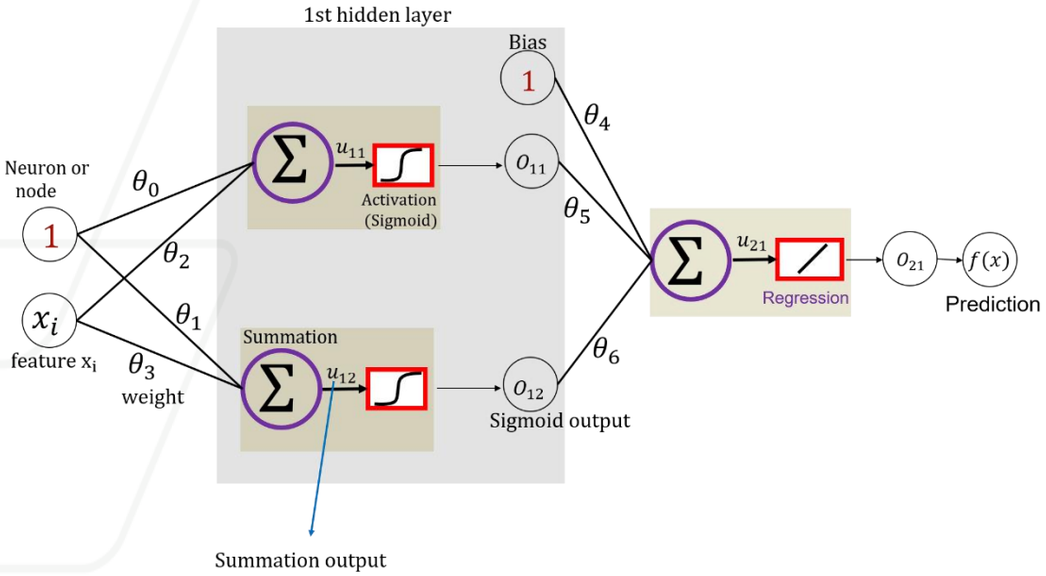
horizontal shrinking or stretchy

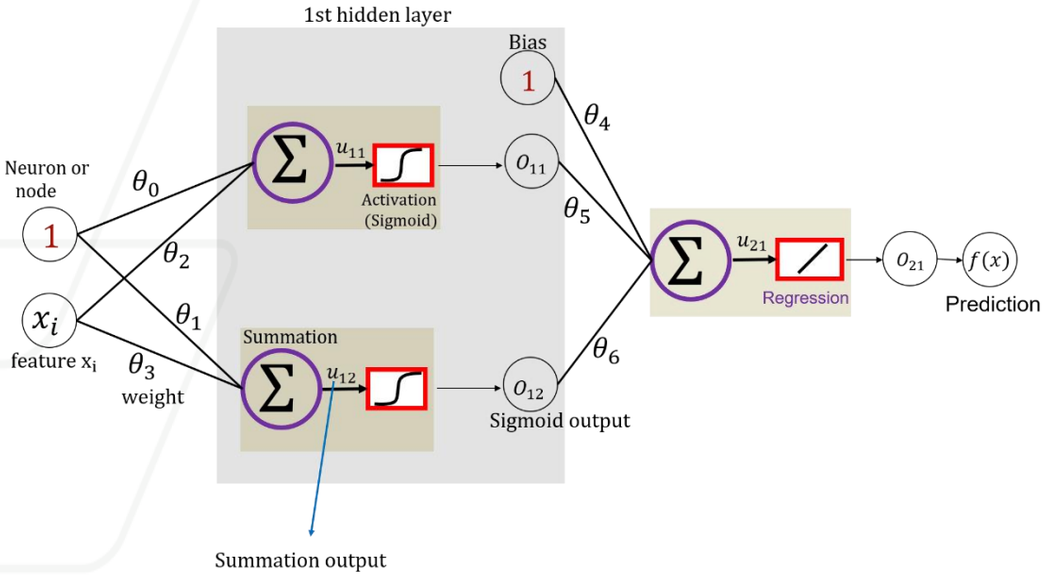
Georgia Tech

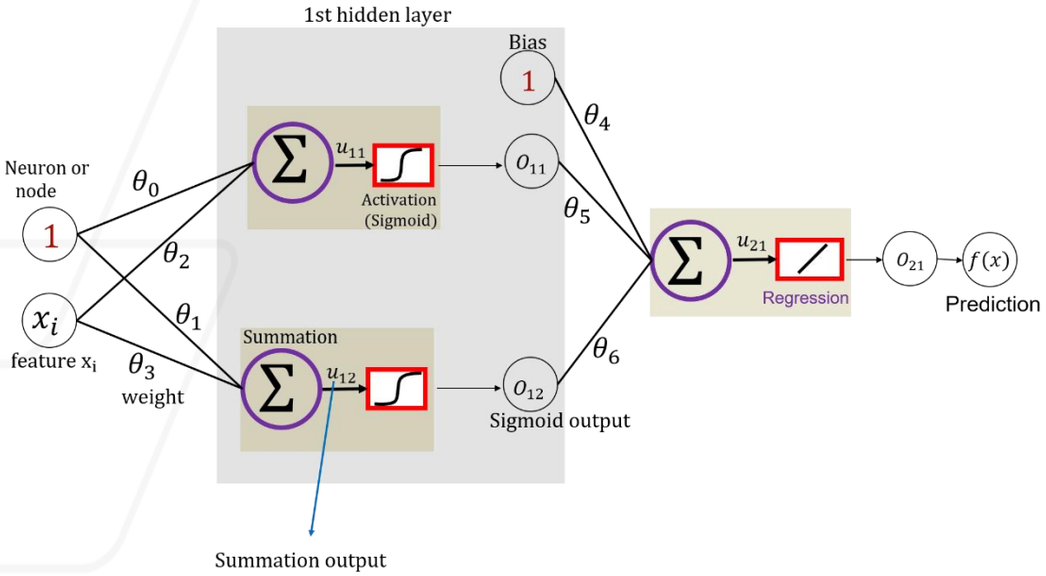
https://playground.tensorflow.org

How Neural Network works?

Neurons:


1st hidden layer
Bias
1
Neuron or
node
1
x_i
feature x_i
θ_0
θ_1
θ_2
θ_3
weight
u_{11}
Activation
(Sigmoid)
o_{11}
Summation
u_{12}
o_{12}
Sigmoid output
Summation output
θ_4
θ_5
θ_6
u_{21}
Regression
o_{21}
$f(x)$
Prediction


1st hidden layer
Bias
1
Neuron or
node
1
x_i
feature x_i
θ_0
θ_1
θ_2
θ_3
weight
u_{11}
Activation
(Sigmoid)
o_{11}
Summation
u_{12}
o_{12}
Sigmoid output
Summation output
θ_4
θ_5
θ_6
u_{21}
Regression
o_{21}
$f(x)$
Prediction


1st hidden layer
Bias
1
Neuron or
node
1
x_i
feature x_i
θ_0
θ_1
θ_2
θ_3
weight
u_{11}
Activation
(Sigmoid)
o_{11}
Summation
u_{12}
o_{12}
Sigmoid output
Summation output
θ_4
θ_5
θ_6
u_{21}
Regression
o_{21}
$f(x)$
Prediction